Fundamentals of Programming using Python • First Year, FinTech

Chapter 03: Variables and the Basic Commands

Names and values • re-assignment • the 35 keywords • swapping • `+=` • `print` options • tracing by hand

Chapter 03 of 10

Dr. Brahim Zirari

Yahia Farès University of Medea

in linkedin.com/in/brahimzirari · Google Scholar

October 2026

1. What a variable really is

❑ **Not a box** – The usual picture — a variable as a box holding a value — is convenient but wrong, and it will mislead you in Chapter 05. A **name** in Python is a *label attached to a value*. The value exists; the name points at it.

Two names, two values. The name is a label, not a box.

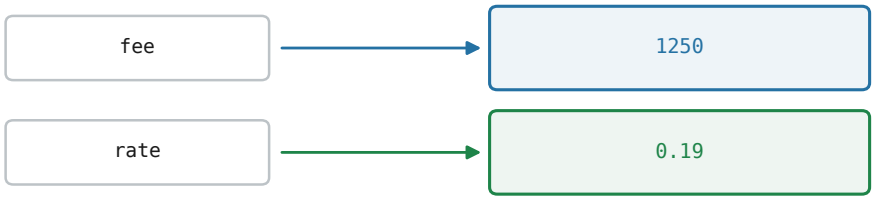


Figure 1. `fee = 1250` attaches the label `fee` to the value 1250.

❑ **Why the difference matters** – Assignment never copies the value. It moves the label. When you write `fee = 1400`, the value 1250 is not modified — the name `fee` simply now points somewhere else.

After fee = 1400: the name moved, the old value stayed where old_fee points.

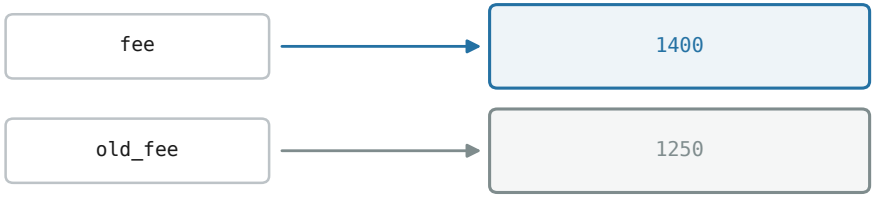


Figure 2. After `old_fee = fee` and then `fee = 1400`: two names, two values, no copying.

2. Assignment and re-assignment

❑ **The rule** – In `name = expression`, the **right side is computed first**, then the name is attached to the result. Always read an assignment right-to-left.

```
total = 100
total = total + 25
print(total)
```

Output

```
125
```

❑ **Line 2 is not a contradiction** – As mathematics, $t = t + 25$ has no solution. As Python it is an instruction: *take the current value of* `total`*, add 25, and attach the name* `total` *to the result.* The old value is simply forgotten.

❑ **A name has no fixed type** – Python is **dynamically typed**: the type belongs to the value, not to the name. The same name may hold a number on one line and text on the next.

Pitfall: this freedom is a trap as much as a convenience. If `total` is a number in one half of your program and text in the other, every calculation using it becomes a guess. Decide what a name holds, and keep it to that.

❑ **Using a name before it exists** – Python reads your file top to bottom. A name must be assigned on an *earlier* line than the one that uses it, or you get the `NameError` from Chapter 01.

3. Names: the rules

❑ **Enforced by Python** –

- Letters, digits and the underscore `_` only. No spaces, no hyphens, no accents.
- It may not *begin* with a digit. `rate2` is fine, `2rate` is a `SyntaxError`.
- Case matters: `Rate`, `rate` and `RATE` are three different names.
- It may not be one of the 35 reserved keywords below.

A name may be re-assigned at any time, even to another type.

```
>>> total = 100

>>> total = total + 25

>>> total
125

>>> total = 'one hundred'

>>> type(total)
<class 'str'>
```

Figure 3. Re-assignment in the interpreter. Nothing stops a name changing type.

❑ **The 35 keywords** – These words belong to the language. Using one as a name is a `SyntaxError`. You do not need to learn the list — your editor colours them — but you should recognise one when you see it.

`False`	`None`	`True`	`and`	`as`
`assert`	`async`	`await`	`break`	`class`
`continue`	`def`	`del`	`elif`	`else`
`except`	`finally`	`for`	`from`	`global`
`if`	`import`	`in`	`is`	`lambda`
`nonlocal`	`not`	`or`	`pass`	`raise`
`return`	`try`	`while`	`with`	`yield`

❑ **Conventions, not rules** – Python will accept anything legal. Other people — and you, later — will not.

Style	Use it for	Example
`snake_case`	ordinary variables	`interest_rate`, `total_due`
`UPPER_CASE`	a value that must never change	`VAT_RATE = 0.19`
`_name`	something internal, not for the reader	`_tmp`

Pitfall: three names to avoid even though Python allows them: `l` (lowercase L), `I` (capital i) and `O` (capital o). In most fonts they are indistinguishable from `1` and `0`.

Remark: a name you can read aloud is worth more than a short one. `monthly_payment` costs you nine extra characters once and saves a reader a guess every time it appears.

4. Shadowing: a quiet mistake

❑ **Python will let you** – Function names are not keywords, so nothing stops you writing `print = 5` or `str = "x"`. The line runs. What breaks is the *next* use of the real function.

```
sum = 1250 + 400
print(sum)        # fine, prints 1650
# ... fifty lines later ...
# sum([1, 2, 3])  <- TypeError: int is not callable
```

Pitfall: the error appears far from the line that caused it, which is what makes this hard to find. Never name a variable `sum`, `list`, `str`, `type`, `input`, `max` or `min`. If your editor colours a name differently from your other variables, that is the warning.

5. Multiple assignment and swapping

❑ **Several names at once** – Python allows one line to assign several names, by position.

```
city, revenue, units = "Medea", 184500, 320
print(city, revenue, units)
```

Output

```
Medea 184500 320
```

❑ **The same value to several names** –

```
x = y = z = 0
print(x, y, z)
```

Output

```
0 0 0
```

❑ **Swapping** – In most languages exchanging two values needs a third variable. In Python it is one line, and this is the form you should use.

```
a = 10
b = 25
a, b = b, a
print(a, b)
```

Output

```
25 10
```

Remark: it works because the whole right side is evaluated before *anything is assigned. Python builds the pair `(25, 10)` first, then attaches the names. Written the naive way — `a = b` then `b = a` — both names end up holding 25, and the 10 is lost.*

6. Augmented assignment

❑ **A shorter form** – `total = total + 25` can be written `total += 25`. Every arithmetic operator has such a form.

Short	Means	Typical use
`t += 25`	`t = t + 25`	add to a running total
`t -= 25`	`t = t - 25`	take a fee off a balance
`t *= 1.19`	`t = t * 1.19`	apply a rate
`t /= 4`	`t = t / 4`	split
`t //= 4`	`t = t // 4`	split, whole part only
`t %= 4`	`t = t % 4`	keep the remainder
`t **= 2`	`t = t ** 2`	square

```
balance = 50000
balance -= 1250      # a fee
balance += 18400     # a deposit
print(balance)
```

Output

```
67150
```

Pitfall: `+=` requires the name to exist already. `total += 25` with no earlier `total` is a `NameError`, because Python must read the old value before it can add to it. A running total must be started at 0 on a line of its own.

7. `print()` in full

❑ **More than one value** – `print` accepts any number of values, separated by commas, and puts **one space** between them.

```
print("Medea", 184500, "DZD")
```

Output

```
Medea 184500 DZD
```

❑ **Two options** – `sep` changes what goes *between* the values; `end` changes what goes *after* the last one. Both are written as `name=value` inside the brackets.

```
print("2026", "10", "15", sep="-")
print("Loading", end="")
print("... done")
```

Output

```
2026-10-15
Loading... done
```

Remark: the default is `sep=" "` and `end="\n"` — one space between, a new line after. Setting `end=""` is how you make two `print` calls write a single line.

❑ **Three useful escapes inside text** –

`\n`	a new line
`\t`	a tab — lines up columns in a small table
`\"`	a double quote inside double-quoted text

```
print("City\tRevenue")
print("Medea\t184500")
```

Output

```
City    Revenue
Medea   184500
```

8. `input()` in full

❑ **One argument, one result** – `input("message")` prints the message, waits, and returns **text**. The message is optional but you should always give one: a program that waits with a blank screen looks broken.

```
name = input("Client name: ")
units = int(input("Units sold: "))
price = float(input("Unit price in DZD: "))

print(f"{name}: {units * price:.2f} DZD")
```

Pitfall: `int(input(...))` fails with a `ValueError` if the user types anything that is not a whole number — including `12.5`, a space, or nothing at all. Chapter 08 gives you the tools to check first. Until then, assume a cooperative user.

9. The read–compute–show shape

❑ **Almost every small program** – has the same five steps. Writing them in this order, with one job per line, is most of what makes a program readable.

10. The basic commands

11. Tracing a program by hand

❑ **The one skill that repays itself** – Before running a program, work through it with a pen: one row per line, one column per variable, writing the new value after each line. If your trace and the output disagree, you have just found either a bug or a misunderstanding — both worth having.

The program

```
1  capital = 400000
2  fee = 1250
3  capital -= fee
4  capital, fee = fee, capital
5  print(capital, fee)
```

Line	capital	fee	What happened
1	400000	—	the name is created
2	400000	1250	a second name
3	398750	1250	400000 - 1250
4	1250	398750	both swapped in one step
5	1250	398750	prints `1250 398750`

Remark: line 4 is where a hand trace earns its place. Written as two separate assignments it would give `1250 1250`; written as a swap it does not. The table makes the difference visible before the program runs.

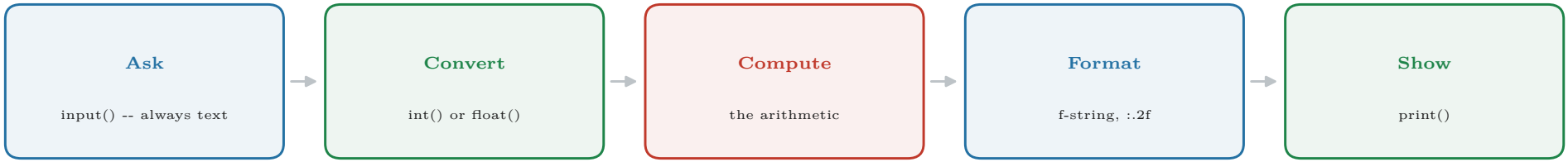


Figure 4. The shape of a small program. Keep the five steps separate.

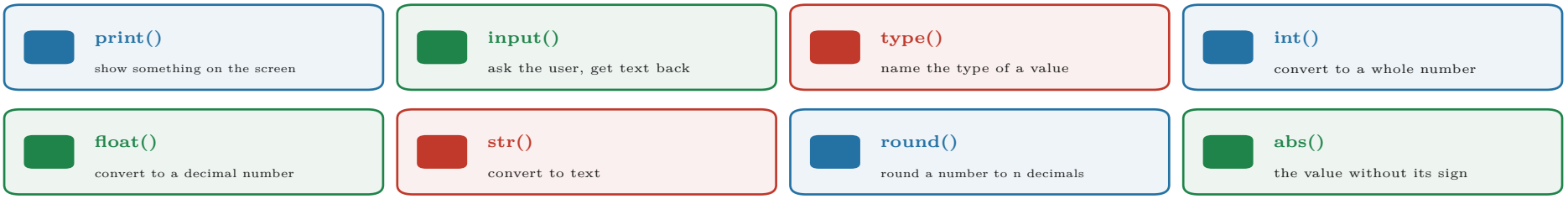


Figure 5. The eight commands you can use so far. Everything in the TP is built from these.

Quick reference

Assignment	
`n = v`	compute `v`, attach the name `n` to it
`a, b = 1, 2`	two names in one line, by position
`a, b = b, a`	swap. The right side is built first
`x = y = 0`	the same value, two names
`t += 25`	`t = t + 25`. `t` must already exist
Names	
legal	letters, digits, `_`; never starting with a digit
`snake_case`	ordinary variables
`UPPER_CASE`	a value that must not change
avoid	`sum list str type input max min`, and `l I O`
`print`	
`print(a, b)`	one space between, a new line after
`sep="-"`	change what goes between the values
`end=""`	change what goes after; `""` means stay on the line
`\n \t`	new line, tab
`input`	
`input("msg")`	returns text, always
`int(input(...))`	whole number; `ValueError` if not one

What you can do after this chapter

- Explain why a name is a label and not a box, and why assignment never copies.
- Read any assignment right-to-left, including `t = t + 25`.
- Name a variable legally and conventionally, and recognise a keyword.
- Say why `sum = 1650` is dangerous and where the error will appear.
- Assign several names in one line, and swap two values.
- Use `+=` and the rest, and say why the name must exist first.
- Control `print` with `sep` and `end`.
- Trace a program by hand and compare the trace with the real output.

❑ **Next** – Chapter 04 — the values themselves: numbers, strings and booleans.

↪ style guide, PEP 8: peps.python.org/pep-0008/

↪ the keyword list: docs.python.org/.../lexical_analysis.html